MILESTONE 3:

Step 1: Implement the Consumer Behavior Intelligence Engine

1. Data Ingestion & Trajectory Analysis

- **Ingest Session Logs:** Pull shopper session data from video processing pipelines (e.g., shopper IDs, entry/exit timestamps, (x, y) coordinates per frame, zone visit logs).
- **Calculate Journey Metrics:**
    - **Total Path Distance:** Aggregate sequential Euclidean distances between tracked (x, y) coordinates.
    - **Zone Dwell Times:** Sum duration spent inside bounding polygons defining specific store zones.
    - **Movement Velocity:** Calculate speed variations to distinguish between browsing, stopping, and passing through.

2. Shopper Segmentation Classification

Implement a classification module using Scikit-learn (e.g., K-Means clustering or rule-based heuristics) to map session patterns into customer segments:

- **Explorers:** High total path distance, high dwell time across multiple zones, low pickup frequency.
- **Quick Buyers:** Low dwell time, direct path trajectory to a single zone, immediate product pickup and checkout.
- **Comparison Shoppers:** Extended dwell time at a single shelf, high product pickup and return events.
- **Impulse Buyers:** Moderate path length, short view duration followed by immediate pickup.
- **Brand Loyal Customers:** Targeted navigation to specific brand zones with high purchase conversion.

Step 2: Build Heatmap Generation Workflows

1. Coordinate Projection & Planogram Mapping

- Map 2D camera coordinates (x_c, y_c) to normalized 2D store layout / shelf planogram coordinates (x_p, y_p) using homography matrices computed via OpenCV (cv2.findHomography).

2. Spatial Aggregation & Density Estimation

- **Time-Window Aggregation:** Store gaze coordinates and shopper location points in PostgreSQL / MongoDB aggregated by time intervals (e.g., hourly, daily).
- **Kernel Density Estimation (KDE):** Apply Gaussian KDE on spatial data using NumPy and OpenCV to convert discrete interaction points into smooth density heatmaps.

3. Image Rendering & API Delivery

- Render traffic and gaze heatmaps using Plotly/Seaborn or transparent PNG overlays for frontend rendering.
- Build FastAPI endpoints (/api/v1/heatmaps/store, /api/v1/heatmaps/shelf) supporting filters by store ID, shelf ID, date range, and shopper segment.

Step 3: Develop the Product Attractiveness Scoring Engine

1. Feature Extraction & Metric Normalization

Extract aggregated product interaction metrics for a given SKU within a time window:

- **Attention Duration (A):** Total seconds shoppers looked at the product.
- **Interaction Frequency (I):** Total count of physical interactions (touch/hover).
- **Pickup Rate (P):** Ratio of total pickups to total shelf views ($\frac{\text{Pickups}}{\text{Views}}$).
- **Purchase Conversion Rate (C):** Ratio of purchases to total pickups ($\frac{\text{Purchases}}{\text{Pickups}}$).
- **Repeat Engagement Rate (R):** Percentage of unique shoppers who re-engaged with the SKU during a session.

Normalize each raw metric to a scale of $[0, 100]$ using Min-Max scaling across all products in the same category.

2. Weighted Scoring Calculation

Implement the formula to calculate the overall Product Attractiveness Score:

$$\text{Product Attractiveness Score} = (0.35 \times A) + (0.25 \times I) + (0.20 \times P) + (0.15 \times C) + (0.05 \times R)$$

3. Endpoint & Database Persistence

- Write output scores into PostgreSQL (product_attractiveness_scores table).
- Expose endpoints (/api/v1/analytics/attractiveness) to return scores, ranked lists, and historical trends per product and shelf placement.

Step 4: Generate Optimization & Recommendation Engine

1. Build Rule-Based Optimization Logic

Develop heuristic decision trees in Python to evaluate behavioral patterns against thresholds:

- **High Attention + Low Pickup:** Trigger packaging/pricing check (product grabs gaze, but shoppers decline to pick it up).
- **High Pickup + Low Purchase Conversion:** Trigger product quality/pricing inspection (shoppers pick up product, read label, but return it).
- **Cold Shelf Zones:** Detect areas with consistently low traffic/dwell metrics and recommend placing high-performing anchor products there to increase flow.
- **Eye-Level Optimization:** Flag high-attractiveness products currently placed on bottom shelves for relocation to eye-level slots (vertical shelf heights with highest gaze density).

2. Recommendation Engine API Integration

- Format output recommendations into structured JSON (e.g., priority level, target shelf ID, product SKU, action item, expected conversion uplift).
- Store actionable recommendations in the database for display on the store manager and analyst dashboards.

Step 5: Create Retail Intelligence Dashboards:

1. Frontend Setup (React.js / Next.js & Tailwind CSS)

- Set up state management and API clients to stream dashboard data from FastAPI.
- Integrate Chart.js and Plotly for interactive visualization components.

2. Dashboard View Implementation

- **Retail Analyst Dashboard:**
  - Interactive store traffic heatmaps with layer controls (gaze vs. foot traffic).
  - Customer journey sankey/path diagrams.
  - Customer segmentation breakdown charts.
  - Detailed product attractiveness scoring tables with filterable columns.
- **Marketing Manager Dashboard:**
  - Promotional display performance widgets.
  - Campaign visibility and engagement metrics before vs. after placement changes.
  - Conversion rate comparisons across promotional zones.
- **Store Manager Dashboard:**
  - Real-time shelf engagement and traffic anomaly indicators.
  - High-priority shelf optimization recommendations.
  - Daily conversion metrics summary.

OUTCOMES WE have get in the end of milestone-3:

1. Consumer Behavior Engine Operational:

- **Behavior Analysis & Trajectory Pipelines:** Active processing of shopper session metrics (total path distance, zone dwell times, movement velocity, and interaction event counts).
- **Automated Shopper Segmentation:** Operational classification module assigning every shopper session into one of five core segments: *Explorers*, *Quick Buyers*, *Comparison Shoppers*, *Impulse Buyers*, or *Brand Loyal Customers*.
- **Session Analytics Persistence:** Storage of processed journey metrics and behavioral classifications in PostgreSQL/TimescaleDB.

2. Attention Heatmaps Functional

- **2D Planogram Spatial Mapping:** Functional homography coordinate projection transforming camera pixel coordinates (x, y) to normalized store and shelf layouts.
- **Kernel Density Estimation (KDE) Rendering:** Smooth heatmaps generated across four visual layers:
    - Store traffic/movement paths
    - Zone activity density
    - Product gaze focus
    - Shelf interaction hotspots ($N \times M$ grid level)
- **Heatmap Caching & API Services:** FastAPI endpoints serving cached, filterable heatmap matrix data by store, shelf, date range, and segment type.

3. Optimization Recommendation Workflows Completed

- **Product Attractiveness Scoring Pipeline:** Automated batch scoring jobs calculating weighted SKU attractiveness based on the system formula:

  $$\text{Attractiveness Score} = 0.35(A) + 0.25(I) + 0.20(P) + 0.15(C) + 0.05(R)$$

- **Rule-Based Optimization Engine:** Automated diagnostic checks generating real-time merchandising recommendations (e.g., flagging high attention/low pickup products for price reviews, or suggesting bottom-shelf top performers be relocated to eye-level).
- **Structured Recommendation APIs:** Delivery of actionable optimization payloads (including priority level, target SKU, shelf location, and projected conversion impact) directly to the dashboard interfaces.

Technical Understanding:

1. Data_Ingestion (Shopper Profiling)

[Kafka/Redis Stream] ➔ [Kalman Filter] ➔ [Metrics Calculation] ➔ [K-Means ML] ➔ [PostgreSQL]

- Raw Camera Data Stream: Ingests live tracking points as (x, y) coordinates.
- Kalman Filter: Cleans up "jittery" or jumpy movement lines from camera tracks to create smooth paths.
- Metrics Calculation: Measures how far customers walked, how long they paused, and their speed.

- K-Means ML: Automatically classifies each customer into one of 5 shopping types (*Explorer*, *Quick Buyer*, etc.).
- PostgreSQL: Saves all these calculated profiles into the primary database.

2. Heatmap_Engine (Visualizing Hotspots)

[Camera Coordinates (x_c, y_c)] ➔ [findHomography] ➔ [KDE Engine] ➔ [Redis Cache]

- findHomography: Uses OpenCV to convert 2D camera pixel points (x_c, y_c) onto a flat 2D store planogram layout (x_p, y_p).
- Kernel Density Estimation (KDE): Blurs discrete coordinate points into smooth weather-style density maps (red = high activity, blue = low activity).
- Redis Cache: Saves these heatmaps in fast memory so the web application loads them instantly without delay.

3. Attractiveness_Engine (Product Scoring)

[PostgreSQL Metrics] ➔ [Min-Max Scaling] ➔ [Weighted Formula] ➔ [Save Score]

- Fetch Metrics: Pulls product engagement data (Attention Duration A, Interactivity I, Pickups P, Conversions C, Repeat Looks R).
- Min-Max Normalization: Rescales raw metrics to a standard $0–100$ scale across category averages.
- Weighted Formula: Applies the formula:
  text{Score} = (0.35 \times A) + (0.25 \times I) + (0.20 \times P) + (0.15 \times C) + (0.05 \times R)
- Save Score: Stores the final $0–100$ score for every product back into the database.

4. Optimization_Engine (Smart Rule-Based Advice)

[Evaluator] ➔ [Rule Checks (If/Then)] ➔ [JSON Alert Payload]

- Evaluator: Inspects computed product scores alongside shelf location data.
- Rule Checks:
  - *High Attention + Low Pickups*? $\rightarrow$ Suggests pricing or packaging checks.
  - *High Score + Placed on Bottom Shelf*? $\rightarrow$ Suggests moving the item to eye level.
  - *Low Traffic in Aisle*? $\rightarrow$ Suggests placing a popular anchor product nearby.

- JSON Payload: Packages these alerts into structured messages ready for API delivery.

5. UI_Dashboards (Display Layer)

[FastAPI Endpoints] ➔ [React / Next.js Web App] ➔ [3 User Roles]

- FastAPI: Serves data via standard backend endpoints (/api/v1/heatmaps, /api/v1/recommendations, /api/v1/analytics).
- React / Next.js: Displays interactive charts, heatmaps, and recommendation feeds tailored to:
    - Store Managers: Daily operational alerts and shelf relocation recommendations.
    - Retail Analysts: Deep-dive heatmaps, customer movement paths, and category score tables.
    - Marketing Managers: Campaign visibility metrics and promotional display performance.

Step-by-Step Implementation Sequence:

Step 1: Trajectory Analysis & Behavioral Segmentation

1. **Event Consumption:** Ingest continuous stream data (shopper ID, timestamp, (x, y) coordinates, zone interactions) from Kafka or Redis Streams.
2. **Path Normalization:** Smooth noisy coordinate tracking jitter using a Kalman filter.
3. **Metric Calculation:** Calculate total path length, velocity, and time spent inside polygon-defined shelf zones.
4. **Segmentation Classification:** Feed computed features into a Scikit-learn model (K-Means / Decision Trees) to categorize sessions into five buyer personas (*Explorers*, *Quick Buyers*, *Comparison Shoppers*, *Impulse Buyers*, *Brand Loyal Customers*).

Step 2: Spatial Homography & Heatmap Pipeline

1. **Coordinate Transformation:** Calibrate camera pixel coordinates (x_c, y_c) to flat 2D store layout coordinates (x_p, y_p) using cv2.findHomography.

2. **KDE Matrix Rendering:** Run Gaussian Kernel Density Estimation via OpenCV/SciPy over coordinate density maps.
3. **Multi-Layer Generation:** Render distinct heatmaps for overall store traffic, specific zone dwell times, product gaze focus, and grid-level shelf engagement.
4. **Caching Layer:** Cache base64 images or dense arrays directly into Redis for sub-100ms API retrieval.

Step 3: Product Attractiveness Scoring

1. **Feature Aggregation:** Extract Attention Duration (A), Interaction Frequency (I), Pickup Rate (P), Purchase Conversion Rate (C), and Repeat Engagement Rate (R) per SKU from PostgreSQL.
2. **Min-Max Scaling:** Normalize raw feature values to a $[0, 100]$ scale across category benchmarks.
3. **Formula Execution:** Apply the weighted formula: $$\text{Attractiveness Score} = (0.35 \times A) + (0.25 \times I) + (0.20 \times P) + (0.15 \times C) + (0.05 \times R)$$
4. **Automated Batch Processing:** Schedule automated scoring batch tasks (via Celery or APScheduler) every 15–30 minutes.

Step 4: Diagnostic Recommendation Engine

1. **Rule Evaluation:** Pass SKU scores and placement coordinates through heuristic decision trees.
2. **Diagnostic Rules:**
    - High gaze attention with low pickups $\rightarrow$ Output price/packaging check alert.
    - Top-performing score on bottom shelf $\rightarrow$ Output relocation recommendation to eye-level.
    - Low overall traffic density in aisle $\rightarrow$ Output anchor product placement suggestion.
3. **API Delivery:** Return structured actionable JSON payloads containing shelf IDs, SKU IDs, priority level, and expected impact.

Step 5: Dashboard Visualization Layer

1. **FastAPI Endpoints:** Expose RESTful endpoints for heatmaps, metrics, scores, and recommendation feeds.
2. **Frontend UI (React/Next.js & Tailwind CSS):**
    - **Store Manager Dashboard:** Live traffic alerts, operational summary, actionable shelf recommendations.

    - **Retail Analyst Dashboard:** Heatmap canvas overlays, customer journey Sankey charts, shopper segment distribution.
    - **Marketing Manager Dashboard:** Campaign engagement metrics, promotion visibility ratings, conversion lift reports.

3 Main Views:

- **Store Manager View:** Sees real-time alerts (e.g., "Move Product X to Eye Level") and hourly sales/traffic trends.
- **Retail Analyst View:** Sees interactive heatmaps, customer movement diagrams, and raw data charts.
- **Marketing Manager View:** Sees how well special promotional displays or sales ads are attracting eyes.

Intern Task: Connect the React frontend components to the backend **FastAPI** endpoints to display heatmaps and recommendation lists nicely.

Step	Goal	Main Tool / Tech
1. Behavior Engine	Categorize shoppers into 5 segments	Python, Scikit-learn, Pandas
2. Heatmap Workflows	Render red/blue hotspot overlays	OpenCV, SciPy (KDE), Redis
3. Attractiveness Score	Compute $0 - 100$ score per SKU using weighted formula	SQL / PostgreSQL, Python
4. Recommendations	Generate JSON alerts for pricing/shelf changes	Python Rule Engine
5. Dashboards	Build UI views for Managers & Analysts	React, Next.js, FastAPI, Chart.js